Migration guide: from `codefresh-report-image` to standalone CSDP enrichment steps

Who this guide is for

This guide is for you if your CI pipeline currently reports images to Codefresh using either of these:

- The `codefresh-io/codefresh-report-image` GitHub action
- The image reporting and enrichment templates from the GitOps Runtime (the Argo Workflows-based enrichment)

The Argo Workflows that power image enrichment inside the runtime are deprecated. Image reporting keeps working, but it moves out of the runtime and into your CI pipeline as three containers you run yourself. No runtime component is involved anymore.

The result in Codefresh stays the same: your images appear in the Images dashboard with Git, Jira, and pull request metadata attached.

What changes

Before (`codefresh-report-image`)	**After (standalone steps)**
One action or step	Three containers: report (required), Jira enrichment (optional), and Git enrichment (optional)
`CF_RUNTIME_NAME` selects a runtime	Removed — no runtime is used
`CF_CONTAINER_REGISTRY_INTEGRATION` names a Codefresh integration	You pass registry credentials directly (for example, `DOCKERHUB_USERNAME` and `DOCKERHUB_PASSWORD`)
`CF_ISSUE_TRACKING_INTEGRATION` names a Codefresh integration	Pass the same integration name as `JIRA_CONTEXT` , or pass Jira credentials directly (`JIRA_HOST_URL` , `JIRA_EMAIL` , and `JIRA_API_TOKEN`)
`CF_IMAGE`	`IMAGE_NAME` — the same variable name in all three steps
`CF_GIT_BRANCH`	`BRANCH` (Git enrichment step)
`CF_JIRA_MESSAGE`	`JIRA_MESSAGE` (Jira enrichment step)
`CF_JIRA_PROJECT_PREFIX`	`JIRA_PROJECT_PREFIX` (Jira enrichment step)
`CF_GITHUB_TOKEN`	`GITHUB_TOKEN` (Git enrichment step)
`CF_API_KEY`	`CF_API_KEY` — the same key works in all three steps

The three steps

#	Step	Container image	Required
1	Report image	`quay.io/codefreshplugins/argo-hub-codefresh-csdp-report-image-info:1.1.30`	Yes — always runs first
2	Jira enrichment	`quay.io/codefreshplugins/argo-hub-codefresh-csdp-image-enricher-jira-info:1.1.30`	Optional, after step 1
3	Git enrichment	`quay.io/codefreshplugins/argo-hub-codefresh-csdp-image-enricher-git-info:1.1.30`	Optional, after step 1

Step 1 creates the image entity in Codefresh. Steps 2 and 3 annotate that entity, so they must run after step 1 finishes. Steps 2 and 3 don't depend on each other and can run in parallel.

You configure each step entirely through environment variables — there are no CLI arguments. Each step validates its inputs on startup and fails with a clear `ValidationError` message when a required variable is missing or malformed.

Pin the version tag (`1.1.30`) rather than a floating tag, so your pipeline behavior stays predictable.

Before you start

Collect these credentials and store them as secrets in your CI system:

Credential	Where to get it	Used by
Codefresh API key	User settings → API Keys → Generate	All three steps (`CF_API_KEY`)
Registry read credentials	Your registry (for example, a Docker Hub username and access token)	Step 1
Jira API token and account email	Atlassian account → Security → API tokens	Step 2
Git provider token	GitHub: the built-in job token is usually enough	Step 3

Five rules that prevent hard-to-diagnose failures

1. **Run the images with their default entrypoint.** The images are distroless — they contain Node.js and the step code, but no shell. This means they can't run as GitHub Actions job containers (`container:`), Codefresh freestyle steps with `commands` , or Jenkins `docker.image(...).inside` blocks, because all of those need a shell inside the image. Instead, run each step with `docker run` (or an equivalent that uses the image's default entrypoint) and pass configuration as environment variables. The examples below show this pattern for each CI system.
2. **The image URI must be identical in all three steps.** Use the full URI including registry and tag, for example `docker.io/myuser/my-app:1.2.3` , and pass it as `IMAGE_NAME` to every step. If step 2 or 3 receives even a slightly different string, it annotates a nonexistent entity and the data goes nowhere. Define the URI once and reuse it.
3. **`JIRA_HOST_URL` is a full URL.** Include the protocol: `https://mycompany.atlassian.net` . The step validates this value as a URL and fails if the protocol is missing.
4. **GitHub Actions: never pass the image URI through a job output if it contains a secret.** If your image path embeds a secret (for example, `${{ secrets.DOCKERHUB_USERNAME }}`), GitHub silently drops the output (`Skip output since it may contain secret`) and the downstream job receives an empty string. Pass only non-secret parts (like the tag) between jobs, and rebuild the full URI in each job's `env` .
5. **Read the logs, not just the exit code, when `FAIL_ON_NOT_FOUND` is `false` .** With the default setting, the Jira step exits successfully even when no issue key matches your message. Configuration errors and API failures always fail the step, but a quiet "no issues detected" only shows in the logs.

Environment variable reference

Step 1 — report-image-info

Variable	Required	Value
`IMAGE_NAME`	Yes	Full image URI, including registry and tag
`CF_API_KEY`	Yes	Codefresh API key
Registry credentials	Yes — one set	Docker Hub: `DOCKERHUB_USERNAME` + `DOCKERHUB_PASSWORD` · Amazon ECR: `AWS_ACCESS_KEY` + `AWS_SECRET_KEY` + `AWS_REGION` (or `AWS_ROLE`) · Google: `GOOGLE_REGISTRY_HOST` + `GOOGLE_JSON_KEY`, or `GCR_KEY_FILE_PATH` · Other registries: `REGISTRY_DOMAIN` + `REGISTRY_USERNAME` + `REGISTRY_PASSWORD` (add `REGISTRY_INSECURE=true` for HTTP) · Alternatively: `DOCKER_CONFIG_FILE_PATH`
`CF_HOST_URL`	No	Defaults to `https://g.codefresh.io`. Set it only for Codefresh on-premises
`WORKFLOW_NAME`, `WORKFLOW_URL`, `LOGS_URL`	No	Links from the image entity back to your CI run. Point `WORKFLOW_URL` at the specific run (for example, `.../actions/runs/<run-id>` in GitHub Actions) so the link lands on the exact execution that reported the image
`DOCKERFILE_CONTENT`, `DOCKERFILE_PATH`	No	Attach the Dockerfile to the image entity

Note: unlike earlier versions, this step no longer accepts Git metadata (`GIT_BRANCH`, `GIT_REVISION`, and similar). All Git data now comes from the Git enrichment step.

Step 2 — image-enricher-jira-info

Variable	Required	Value
`IMAGE_NAME`	Yes	Exactly the same URI as step 1
`CF_API_KEY`	Yes	The same Codefresh API key
`JIRA_MESSAGE`	Yes	Text to scan for an issue key — typically the branch name or commit message
`JIRA_PROJECT_PREFIX`	Yes	Your Jira project key, for example `CR` (matches `CR-1234` in `JIRA_MESSAGE`)
Jira authentication	Yes — exactly one method	Jira Cloud: `JIRA_API_TOKEN` + `JIRA_EMAIL` + `JIRA_HOST_URL` · Jira Server/Data Center: `JIRA_SERVER_PAT` + `JIRA_HOST_URL` · Codefresh Jira integration: `JIRA_CONTEXT`
`FAIL_ON_NOT_FOUND`	No	`true` fails the step when no issue matches. Defaults to `false`
`CF_HOST_URL`	No	Defaults to `https://g.codefresh.io`

The step validates that you set exactly one of `JIRA_API_TOKEN`, `JIRA_SERVER_PAT`, or `JIRA_CONTEXT`, and fails with a clear message otherwise.

`JIRA_CONTEXT` is the name of a Jira integration configured in your Codefresh account — the same integration `CF_ISSUE_TRACKING_INTEGRATION` referenced before the migration. The step fetches the integration using your `CF_API_KEY` and routes Jira lookups through Codefresh, so you don't need to store Jira credentials in your CI system. If you already have this integration set up, `JIRA_CONTEXT` is the smallest change; otherwise, pass the credentials directly.

Step 3 — image-enricher-git-info

Variable	Required	Value
`IMAGE_NAME`	Yes	Exactly the same URI as step 1
`CF_API_KEY`	Yes	The same Codefresh API key
`GIT_PROVIDER`	Yes	One of `github`, `gitlab`, `bitbucket`, `bitbucket-server`, or `gerrit`
`REPO`	Yes	`owner/repo-name`
`BRANCH`	Yes (except Gerrit)	The pull request's source branch
Provider credentials	Yes	GitHub: `GITHUB_TOKEN` or `GITHUB_CONTEXT` (exactly one) · GitLab: `GITLAB_TOKEN` · Bitbucket: `BITBUCKET_USERNAME` + `BITBUCKET_PASSWORD` · Gerrit: `GERRIT_CHANGE_ID` + `GERRIT_HOST_URL` + `GERRIT_USERNAME` + `GERRIT_PASSWORD`
`GITHUB_API_HOST_URL`	No	Defaults to `https://api.github.com`. Set it for GitHub Enterprise Server
`REVISION`	No	A specific commit SHA to enrich from
`CF_COMMITS_BY_USER_LIMIT`	No	Number of commits to attach. Defaults to 5

Example 1 — GitHub Actions

The workflow below has been validated end to end. A `build` job builds and pushes the image, then three jobs run the CSDP steps with `docker run`.

Secrets are set in the step's `env` block and passed to the container with the `-e VAR` pass-through form, so they never appear in a command line.

```
  csdp-report-image-info:
    runs-on: ubuntu-latest
    needs: [build]
    steps:
      - name: report image info
        env:
          IMAGE_NAME: docker.io/${{ secrets.DOCKERHUB_USERNAME }}/${{ github.event.repository.name }}:${{
needs.build.outputs.version }}
          CF_API_KEY: ${{ secrets.CF_API_KEY }}
          WORKFLOW_NAME: ${{ github.workflow }}
          # Link to this specific run so the link in Codefresh lands on the
          # exact execution that reported the image.
          WORKFLOW_URL: ${{ github.server_url }}/${{ github.repository }}/actions/runs/${{ github.run_id }}
          LOGS_URL: ${{ github.server_url }}/${{ github.repository }}/actions/runs/${{ github.run_id }}
          DOCKERHUB_USERNAME: ${{ secrets.DOCKERHUB_USERNAME }}
          DOCKERHUB_PASSWORD: ${{ secrets.DOCKERHUB_TOKEN }}
        run: |
          docker run --rm \
            -e IMAGE_NAME -e CF_API_KEY \
            -e WORKFLOW_NAME -e WORKFLOW_URL -e LOGS_URL \
            -e DOCKERHUB_USERNAME -e DOCKERHUB_PASSWORD \
            quay.io/codefreshplugins/argo-hub-codefresh-csdp-report-image-info:1.1.30

  csdp-image-enricher-jira-info:
    runs-on: ubuntu-latest
    needs: [build, csdp-report-image-info]
    steps:
      - name: enrich image with jira info
        env:
          IMAGE_NAME: docker.io/${{ secrets.DOCKERHUB_USERNAME }}/${{ github.event.repository.name }}:${{
needs.build.outputs.version }}
          CF_API_KEY: ${{ secrets.CF_API_KEY }}
          JIRA_MESSAGE: ${{ github.head_ref }}
          JIRA_PROJECT_PREFIX: 'CR'
          JIRA_HOST_URL: https://${{ secrets.JIRA_HOST }}
          JIRA_EMAIL: ${{ secrets.JIRA_EMAIL }}
          JIRA_API_TOKEN: ${{ secrets.JIRA_API_TOKEN }}
          FAIL_ON_NOT_FOUND: 'false'
        run: |
          docker run --rm \
            -e IMAGE_NAME -e CF_API_KEY \
            -e JIRA_MESSAGE -e JIRA_PROJECT_PREFIX -e JIRA_HOST_URL \
            -e JIRA_EMAIL -e JIRA_API_TOKEN \
            -e FAIL_ON_NOT_FOUND \
            quay.io/codefreshplugins/argo-hub-codefresh-csdp-image-enricher-jira-info:1.1.30

  csdp-image-enricher-github-info:
    runs-on: ubuntu-latest
    needs: [build, csdp-report-image-info]
    steps:
      - name: enrich image with github info
        env:
          IMAGE_NAME: docker.io/${{ secrets.DOCKERHUB_USERNAME }}/${{ github.event.repository.name }}:${{
needs.build.outputs.version }}
          CF_API_KEY: ${{ secrets.CF_API_KEY }}
          GIT_PROVIDER: github
```

```
          BRANCH: ${{ github.head_ref }}
          REPO: ${{ github.repository }}
          GITHUB_TOKEN: ${{ github.token }}
        run: |
          docker run --rm \
            -e IMAGE_NAME -e CF_API_KEY \
            -e GIT_PROVIDER -e BRANCH -e REPO -e GITHUB_TOKEN \
            quay.io/codefreshplugins/argo-hub-codefresh-csdp-image-enricher-git-info:1.1.30
```

Tips for GitHub Actions:

- Trigger on `pull_request` with `types: [closed]` and gate the build job with `if: github.event.pull_request.merged == true`. This runs the pipeline once per merged pull request while keeping pull request context (like `github.head_ref`) available. A plain `push` trigger loses that context.
- Remember rule 4 about job outputs and secrets.

Example 2 — Codefresh pipeline (classic CI)

Because the images have no shell, omit the `commands` attribute — a freestyle step without `commands` runs the image's default entrypoint, which is exactly what these steps need. Set the shared image URI once as a pipeline variable to satisfy rule 2.

```
version: "1.0"
stages: [build, report, enrich]

steps:
  build_image:
    title: Build & push
    type: build
    stage: build
    image_name: myuser/my-app
    tag: ${{CF_SHORT_REVISION}}
    registry: docker-hub          # your registry integration for pushing

  report_image_info:
    title: Report image to Codefresh
    stage: report
    image: quay.io/codefreshplugins/argo-hub-codefresh-csdp-report-image-info:1.1.30
    environment:
      - IMAGE_NAME=docker.io/myuser/my-app:${{CF_SHORT_REVISION}}
      - CF_API_KEY=${{CF_API_KEY}}
      - DOCKERHUB_USERNAME=${{DOCKERHUB_USERNAME}}
      - DOCKERHUB_PASSWORD=${{DOCKERHUB_TOKEN}}

  enrich_jira:
    title: Enrich with Jira issue
    stage: enrich
    image: quay.io/codefreshplugins/argo-hub-codefresh-csdp-image-enricher-jira-info:1.1.30
    environment:
      - IMAGE_NAME=docker.io/myuser/my-app:${{CF_SHORT_REVISION}}
      - CF_API_KEY=${{CF_API_KEY}}
      - JIRA_MESSAGE=${{CF_BRANCH}}
      - JIRA_PROJECT_PREFIX=CR
      - JIRA_HOST_URL=https://mycompany.atlassian.net
      - JIRA_EMAIL=${{JIRA_EMAIL}}
      - JIRA_API_TOKEN=${{JIRA_API_TOKEN}}
      - FAIL_ON_NOT_FOUND=false

  enrich_git:
    title: Enrich with PR info
    stage: enrich
    image: quay.io/codefreshplugins/argo-hub-codefresh-csdp-image-enricher-git-info:1.1.30
    environment:
      - IMAGE_NAME=docker.io/myuser/my-app:${{CF_SHORT_REVISION}}
      - CF_API_KEY=${{CF_API_KEY}}
      - GIT_PROVIDER=github
      - BRANCH=${{CF_BRANCH}}
      - REPO=${{CF_REPO_OWNER}}/${{CF_REPO_NAME}}
      - GITHUB_TOKEN=${{GITHUB_TOKEN}}
```

Store `CF_API_KEY`, `DOCKERHUB_TOKEN`, `JIRA_API_TOKEN`, `GITHUB_TOKEN`, and similar values as encrypted pipeline or project variables.

Example 3 — Jenkins (declarative pipeline)

Because the images have no shell, `docker.image(...).inside` doesn't work — it needs a shell in the container. Run each step with `docker run` instead, passing environment variables with the `-e VAR` pass-through form so credentials stay out of the command line.

```
pipeline {
  agent any

  environment {
    IMAGE_FULL = "docker.io/myuser/my-app:${env.GIT_COMMIT.take(7)}"
  }

  stages {
    stage('Build & push') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub', usernameVariable: 'DH_USER', passwordVariable:
'DH_PASS')]) {
          sh '''
            echo "$DH_PASS" | docker login -u "$DH_USER" --password-stdin
            docker build -t "$IMAGE_FULL" .
            docker push "$IMAGE_FULL"
          '''
        }
      }
    }

    stage('Report image to Codefresh') {
      steps {
        withCredentials([
          string(credentialsId: 'cf-api-key', variable: 'CF_API_KEY'),
          usernamePassword(credentialsId: 'dockerhub', usernameVariable: 'DOCKERHUB_USERNAME', passwordVariable:
'DOCKERHUB_PASSWORD')
        ]) {
          sh '''
            IMAGE_NAME="$IMAGE_FULL" docker run --rm \
              -e IMAGE_NAME -e CF_API_KEY \
              -e DOCKERHUB_USERNAME -e DOCKERHUB_PASSWORD \
              quay.io/codefreshplugins/argo-hub-codefresh-csdp-report-image-info:1.1.30
          '''
        }
      }
    }

    stage('Enrich - Jira') {
      steps {
        withCredentials([
          string(credentialsId: 'cf-api-key', variable: 'CF_API_KEY'),
          string(credentialsId: 'jira-api-token', variable: 'JIRA_API_TOKEN')
        ]) {
          sh '''
            IMAGE_NAME="$IMAGE_FULL" \
            JIRA_MESSAGE="$BRANCH_NAME" \
            JIRA_PROJECT_PREFIX="CR" \
            JIRA_HOST_URL="https://mycompany.atlassian.net" \
            JIRA_EMAIL="jira-bot@mycompany.com" \
            FAIL_ON_NOT_FOUND="false" \
            docker run --rm \
              -e IMAGE_NAME -e CF_API_KEY \
              -e JIRA_MESSAGE -e JIRA_PROJECT_PREFIX -e JIRA_HOST_URL \
              -e JIRA_EMAIL -e JIRA_API_TOKEN -e FAIL_ON_NOT_FOUND \
              quay.io/codefreshplugins/argo-hub-codefresh-csdp-image-enricher-jira-info:1.1.30
```

```
          '''
        }
      }
    }

    stage('Enrich - Git/PR') {
      steps {
        withCredentials([
          string(credentialsId: 'cf-api-key', variable: 'CF_API_KEY'),
          string(credentialsId: 'github-token', variable: 'GITHUB_TOKEN')
        ]) {
          sh '''
            IMAGE_NAME="$IMAGE_FULL" \
            GIT_PROVIDER="github" \
            BRANCH="$BRANCH_NAME" \
            REPO="myorg/my-app" \
            docker run --rm \
              -e IMAGE_NAME -e CF_API_KEY \
              -e GIT_PROVIDER -e BRANCH -e REPO -e GITHUB_TOKEN \
              quay.io/codefreshplugins/argo-hub-codefresh-csdp-image-enricher-git-info:1.1.30
          '''
        }
      }
    }
  }
}
```

Verify the migration worked

1. The report step's logs should end with `image reported successfully`, and the `image_link` output should contain a link to the image entity in Codefresh.
2. The Jira step's logs should show `detected issues: [<KEY>]` followed by `codefresh assigned issue <KEY> to your gitops image <uri>`.
3. The Git step's logs should show `image patched` followed by `gitops annotation created`, including your pull request number and URL.
4. Open the Images dashboard — your image should show the Git commit, the Jira issue, and the pull request.

Troubleshooting

Symptom	Cause	Fix
`exec: "sh": executable file not found in $PATH`	The image is running as a job container, a freestyle step with `commands`, or a Jenkins `inside` block	Run the image with `docker run` and its default entrypoint (rule 1)
`ValidationError: "IMAGE_NAME" is required` (or any other variable)	The environment variable arrived empty	Check how the value reaches the step — in GitHub Actions, see rule 4
`ValidationError: "JIRA_HOST_URL" must be a valid uri`	The protocol is missing	Include `https://` (rule 3)
`ValidationError` mentioning `JIRA_CONTEXT`, `JIRA_API_TOKEN`, and `JIRA_SERVER_PAT`	More than one, or none, of the Jira authentication methods is set	Set exactly one method (see the step 2 table)
The Jira step succeeds but no issue appears on the image	`JIRA_MESSAGE` doesn't contain `<PREFIX>-<number>`, and `FAIL_ON_NOT_FOUND` is `false`	Pass a branch name or commit message containing the issue key, for example `CR-1234-my-fix`. Check the logs for `detected issues` (rule 5)
`The image you are trying to enrich ... does not exist`	The image URI differs between steps	Make the URI identical everywhere (rule 2)
`401` or authentication errors from Codefresh	An invalid or wrong-account `CF_API_KEY`	Regenerate the key in User settings
Registry errors from the report step	Missing or invalid registry credentials	Provide one complete credential set (see the step 1 table)

Upgrading from the 0.0.6 images

If you followed an earlier version of this guide that used the `0.0.6` images, three things changed:

1. **New image names and tags.** Use `quay.io/codefreshplugins/argo-hub-codefresh-csdp-<step>:1.1.30` instead of the `argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-<step>:main` naming.
2. **No more Jira client patch.** The 1.1.30 Jira step replaced its Jira client library, which fixes the bug where Atlassian's CDN rejected requests with a `403` error. Remove the `sed` workaround from your pipeline.
3. **Renamed variables.** Update these environment variables:

Step	0.0.6 name	1.1.30 name
Report	`IMAGE_URI`	`IMAGE_NAME`
Report	`CF_HOST`	`CF_HOST_URL` (now optional)
Report	`DOCKER_USERNAME` / `DOCKER_PASSWORD`	`DOCKERHUB_USERNAME` / `DOCKERHUB_PASSWORD`
Report	`DOMAIN` / `USERNAME` / `PASSWORD` / `INSECURE`	`REGISTRY_DOMAIN` / `REGISTRY_USERNAME` / `REGISTRY_PASSWORD` / `REGISTRY_INSECURE`
Report	`GIT_BRANCH`, `GIT_REVISION`, `GIT_COMMIT_MESSAGE`, `GIT_COMMIT_URL`, `GIT_SENDER_LOGIN`	Removed — use the Git enrichment step
Jira	`IMAGE`	`IMAGE_NAME`
Jira	`MESSAGE`	`JIRA_MESSAGE`
Jira	`JIRA_HOST` (bare hostname)	`JIRA_HOST_URL` (full URL with `https://`)
Git	`IMAGE_SHA`	`IMAGE_NAME`
Git	—	`GIT_PROVIDER` (new, required)
Git	`GITHUB_API`	`GITHUB_API_HOST_URL`
Git	`GIT_PROVIDER_NAME`	`GITHUB_CONTEXT`

Reference

- Upstream step documentation: report-image-info · image-enricher-jira-info · image-enricher-git-info

> Note: the upstream documentation describes `*_SECRET` and `*_SECRET_KEY` variants for credentials. Those apply to Kubernetes secrets in Argo Workflows and don't apply when you run the containers in CI. Pass the values directly as environment variables.